

**CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CAMPUS DIVINÓPOLIS
GRADUAÇÃO EM ENGENHARIA MECATRÔNICA**

GETÚLIO ANDRADE DA SILVA

**PLATAFORMA INTEGRADA PARA ENSINO DE AUTOMAÇÃO E ROBÓTICA:
INTEGRAÇÃO ENTRE WEBOTS, CODESYS E INDUSOFT**

DIVINÓPOLIS

2026

GETÚLIO ANDRADE DA SILVA

**PLATAFORMA INTEGRADA PARA ENSINO DE AUTOMAÇÃO E ROBÓTICA:
INTEGRAÇÃO ENTRE WEBOTS, CODESYS E INDUSOFT**

Monografia de Projeto Final de Curso apresentado ao Centro Federal de Educação Tecnológica de Minas Gerais, Campus Divinópolis, como parte dos requisitos para conclusão do curso de graduação em Engenharia Mecatrônica.

Orientador: Prof. Me. Marlon Henrique Teixeira

DIVINÓPOLIS

2026

Inserir Ficha Catalográfica

Inserir a Ficha Catalográfica após a disponibilização da mesma.

GETÚLIO ANDRADE DA SILVA

**PLATAFORMA INTEGRADA PARA ENSINO DE AUTOMAÇÃO E ROBÓTICA:
INTEGRAÇÃO ENTRE WEBOTS, CODESYS E INDUSOFT**

Monografia de Projeto Final de Curso apresentado ao Centro Federal de Educação Tecnológica de Minas Gerais, Campus Divinópolis, como parte dos requisitos para conclusão do curso de graduação em Engenharia Mecatrônica.

DIVINÓPOLIS, 01 de Julho de 2026.

Banca Examinadora:

Prof. Me. Marlon Henrique Teixeira
Orientador

Prof. Me. Claudio Henrique Gomes
Convidado

Prof. Dr. Alan Mendes Marotta
Convidado

DEDICO ESTE TRABALHO À MINHA MÃE, QUE
SEMPRE ME APOIOU EM TODAS AS SITUAÇÕES.

AGRADECIMENTOS

Agradeço,

À minha mãe, Eleusa da Silva, que sempre que sempre esteve comigo nos melhores e piores momentos.

À minha família, que me apoiou e ofereceu ajuda para o que fosse necessário.

À todos os meus professores e professoras da instituição, que compartilharam comigo seus conhecimentos e me permitiram estar realizando este projeto.

Ao professor Renato de Souza Dâmaso e ao meu orientador Marlon Henrique Teixeira, que formaram a minha base de conhecimentos para este trabalho.

À todos meus colegas e amigos que compartilharam esta intuição de ensino comigo.

*"Vencer a preguiça é a primeira coisa
que o homem deve procurar se quiser
ser dono do seu destino"*

Thomas Atkinson

RESUMO

O projeto visa à união de três softwares diferentes para criar a representação de um ambiente de automação, ao mesmo tempo que fornece recursos visuais e práticos para facilitar o aprendizado dos alunos na área de automação envolvida. Para isso, o sistema conta com a integração do Webots, como um simulador robótico, que permite a criação de robôs para serem controlados; o CODESYS, como um ambiente de programação de CLPs, possibilitando o controle de outras plataformas com as linguagens já integradas no ambiente; e com o InduSoft, como uma interface gráfica e supervisora para o sistema, possibilitando o acompanhamento em tempo real e a modificação de variáveis do ambiente representado.

A atividade apresentará o controle de um robô exemplo no ambiente Webots, sendo feito por meio da comunicação de dados entre plataformas com o CODESYS. Para tanto, será aplicado o protocolo industrial ModBus, escrito na linguagem Python, sendo versátil, e por possuir uma biblioteca de rede que facilita a integração com a API do software. Também serão empregadas as normas dos CLPs com as linguagens definidas pela IEC 61131-3. Posteriormente, será realizada a integração com o InduSoft, responsável pela camada de supervisão, por meio do protocolo OPC UA, permitindo que o usuário controle o robô e monitore a telemetria por meio de um dashboard.

O foco principal é unir tais diferentes softwares e proporcionar uma ligação entre os conhecimentos adquiridos no início e no final do curso. Facilitando assim a aprendizagem por parte dos alunos, que poderão se utilizar dos arquivos do projeto para, durante as aulas de automação, aprender Linguagem Ladder e Texto Estruturado, linguagens importantes na indústria de automação. Tendo como base um ambiente do Webots que já viram e tiveram a oportunidade de trabalhar anteriormente.

Palavras-chaves: Aprendizagem de automação; Integração de softwares; Modbus TCP; OPC UA.

ABSTRACT

The project aims to combine three different software programs to create a representation of an automation environment, while also providing visual and practical resources to facilitate students' learning in the area of automation involved. To achieve this, the system has the integration of Webots, as a robotic simulator, which allows the creation of robots to be controlled; CODESYS, as a PLC programming environment, enabling the control of other platforms with languages already integrated into the environment; and with InduSoft, as a graphical and supervisory interface for the system, enabling real-time monitoring and modification of variables in the represented environment.

The activity will present the control of an example robot in the Webots environment, being done through data communication between platforms with CODESYS. To this end, the ModBus industrial protocol will be applied, written in the Python language, which is versatile and has a network library that facilitates integration with the software API. PLC standards will also be used with the languages defined by IEC 61131-3. Subsequently, integration will be carried out with InduSoft, responsible for the supervision layer, through the OPC UA protocol, allowing the user to control the robot and monitor telemetry through a dashboard.

The main focus is to unite such different software and provide a link between the knowledge acquired at the beginning and end of the course. This makes learning easier for students, who can use the project files to, during automation classes, learn Ladder Language and Structured Text, important languages in the automation industry. Based on a Webots environment that they have seen and had the opportunity to work with previously.

Keywords: Automation learning; Software integration; Modbus TCP; OPC UA.

LISTA DE ILUSTRAÇÕES

Figura 2.1	Logo do Webots e seu Espaço de Simulação.	19
Figura 2.2	Logo do Codesys e seu Ambiente de Desenvolvimento.	20
Figura 2.3	Logo do InduSoft e sua Área de Trabalho.	21
Figura 2.4	Protocolo de trabalho do Modbus.	23
Figura 2.5	Aruquitetura do sistema OPC.	24
Figura 3.1	Modelagem do Carrinho Projeto Sementes 2020.	26
Figura 3.2	Modelagem do Carrinho PFC 2026.	27
Figura 3.3	Árvore de Cenas 2020.	28
Figura 3.4	Árvore de Cenas 2026.	28
Figura 3.5	Configuração basica dos dispositivos Modsbus.	29
Figura 3.6	IP e Porta utiliziados.	29

LISTA DE TABELAS

LISTA DE ABREVIATURAS E SIGLAS

LISTA DE SÍMBOLOS

SUMÁRIO

1	INTRODUÇÃO	15
1.1	Definição do problema	15
1.2	Objetivos	17
1.3	Contribuições	17
1.4	Organização do trabalho	18
2	DESENVOLVIMENTO	19
2.1	Simuladores	19
2.1.1	Webots	19
2.1.2	Codesys	20
2.1.3	InduSoft	21
2.2	Comunicação	22
2.2.1	Modbus TCP	22
2.2.2	OPC UA	23
2.3	Máquina	24
3	PROJETOS E DESENVOLVIMENTOS METODOLÓGICOS	26
3.1	Modelagem do Robô no Webots	26
3.2	Comunicação com Codesys	28
4	APLICAÇÕES E RESULTADOS	30
5	CONSIDERAÇÕES FINAIS	31
5.1	Conclusão	31
5.2	Sugestões de continuidade	31
	REFERÊNCIAS BIBLIOGRÁFICAS	32
	REFERÊNCIAS BIBLIOGRÁFICAS	32
	APÊNDICE A - TÍTULO DO APÊNDICE A	34
	Primeira Seção	34
	Subseção	34
	ANEXO A - CODIGOS DO PROJETO	35
	Codigo C - Modelagem Webots	35

1 INTRODUÇÃO

O propósito deste documento é consolidar o progresso técnico e o cumprimento das metas estipuladas para o Projeto de Fim de Curso I (PFC I). O escopo desta pesquisa concentra-se na estruturação e validação de uma arquitetura de Comissionamento Virtual, atuando como ferramenta pedagógica por meio da convergência entre o ambiente dinâmico do Webots, a unidade de controle lógico computacional do CODESYS e a interface de supervisão industrial do InduSoft Web Studio.

As atividades práticas conduzidas até o presente momento focam na integração Modbus TCP entre os ambientes de simulação robótica e o ambiente de controle industrial.

1.1 Definição do problema

No ensino de Engenharia Mecatrônica, tem-se uma apresentação abrupta dos conhecimentos relevantes à automação industrial, sem que eles se liguem a qualquer outra área demonstrada e/ou em outros períodos de ensino.

Enquanto o contato inicial com simuladores permite explorar a física e a cinemática robótica com facilidade, o aprendizado posterior de controladores lógicos programáveis (CLPs) muitas vezes ocorre de forma limitada pela oferta limitada de bancadas físicas. Com isso, são utilizados softwares programáveis em uma tentativa de disponibilizar uma maior gama de recursos aos alunos, contudo, a complexidade inerente ao conteúdo e a abordagem isolada desses sistemas representam apenas uma fração da realidade da automação industrial.

O problema central reside na dificuldade de validar lógicas complexas de programação (norma IEC 61131-3) em cenários dinâmicos e realistas sem os custos e riscos de danos ao hardware real. Segundo [Scheifele et al., 2019], o uso de modelos digitais para testes de controle, também chamado de Comissionamento Virtual, é uma etapa essencial da Indústria 4.0 que ainda é pouco explorada no ambiente acadêmico como ferramenta pedagógica integrada.

Para a aplicação de tais modelos, o *Webots* foi escolhido como um simulador robótico que fornece um ambiente de desenvolvimento completo para modelar, programar e simular robôs em um mundo 3D com física realista. Ao mesmo tempo que ele já é explorado dentro da grade de ensino do curso de Engenharia Mecatrônica, ele permite a criação de modelos cinemáticos complexos e a simulação de sensores e atuadores, sendo usado como planta de campo, onde o comportamento dinâmico do robô é validado antes de qualquer implementação em hardware real.

O controlador do robô passaria a ser o software relevante dos estudos de automação na grade curricular: *CODESYS*, um programa de automação industrial, baseado na norma internacional IEC 61131-3 [Marcos et al., 2008][Lewis, 1998] para programação de controladores lógicos. Funcionando como um SoftPLC, que permite o desenvolvimento de lógicas de controle em linguagens como Ladder (LD) e Texto Estruturado (ST). Exercendo a função de unidade de processamento central, sendo responsável por coordenar as ações do robô e gerenciar o intertravamento da célula automatizada.

Seguindo o conteúdo de automação do curso vigente, para a criação de uma interface simplificada, o *InduSoft Web Studio* será o sistema de monitoramento e controle de supervisão e aquisição de dados, também sendo uma Interface Homem-Máquina. Permitindo a criação de dashboards gráficos para a visualização de variáveis de processo e a intervenção remota do operador no sistema. O programa possibilita que o usuário monitore a situação do robô ou atuador e envie comandos de movimentação em tempo real por meio de uma interface profissional.

Sendo assim, a ligação entre os diferentes conteúdos novos e os já existentes, formulada a partir de experiências prévias dos alunos, auxilia numa melhor taxa de aprendizagem para eles. De acordo com [Magrin et al., 2020], a utilização de projeções virtuais (gêmeos-digitais) como um robô virtual para representar sua versão real é totalmente válida e aceitável para a realização de projetos acadêmicos e industriais. [Michel, 2004] também corrobora a utilização do Webots, um software que permite a criação de robôs em 3D que representam suas contrapartes reais, em pesquisas e trabalhos acadêmicos e industriais.

Nesse contexto, a ausência de uma plataforma que integre o sistema robótico Webots, a uma unidade de processamento industrial CODESYS, e a um sistema supervisor InduSoft, limita a percepção do discente sobre a interoperabilidade de sistemas. Sem essa ponte técnica, o estudante perde a oportunidade de vivenciar a coordenação de uma célula de produção automatizada, em que o PLC não atua apenas como controlador do robô, mas como o elemento central de intertravamento e gestão de dados em tempo real. Dessa forma, a proposta de integração desses softwares visa mitigar o abismo entre a simulação puramente acadêmica e a realidade fabril. Permitindo que o aluno desenvolva competências críticas nos protocolos de comunicação industrial utilizados para integrar os diferentes sistemas, Modbus TCP [WEG, 2024a] e OPC UA [WEG, 2024b], e também supervisão de processos, competências estas amplamente exigidas pelo mercado de trabalho atual.

O modelo robótico de exemplo e principal utilizado neste trabalho baseia-se no protótipo desenvolvido no âmbito do Projeto Sementes (CEFET-MG Divinópolis). O referido robô, originalmente projetado para competições de seguidor de linha e desvio

de obstáculos, teve sua primeira versão virtual validada em simulações Webots [Silva, 2020]. O presente PFC propõe a evolução deste ecossistema através da integração de controle industrial (CODESYS) e supervisão remota (InduSoft).

1.2 Objetivos

O presente trabalho visa realizar a integração entre três principais softwares: Webots, CODESYS e InduSoft. Focando no desenvolvimento e na validação de uma infraestrutura de comissionamento virtual, integrando ferramentas de simulação robótica, controle industrial e sistemas supervisórios para fins pedagógicos e técnicos. Como objetivos específicos, podem ser citados:

- Desenvolver um controlador em linguagem Python para realizar a ponte de dados entre o ambiente cinemático do Webots e os registros do SoftPLC via protocolo Modbus TCP [Rosa et al., 2019].
- Projetar e validar códigos de movimentação para o robô exemplo, utilizando as linguagens Ladder (LD) e Texto Estruturado (ST), em conformidade com a norma IEC 61131-3.
- Analisar a portabilidade do sistema e a utilização dos códigos de controle desenvolvidos ao serem aplicados a diferentes modelos robóticos dentro do ambiente de simulação.
- Configurar um servidor OPC UA que realize a comunicação entre a interface de controle lógica e a interface de monitoramento [Cavalieri and Chiacchio, 2013].
- Configurar uma interface de supervisão industrial, utilizando o InduSoft Web Studio, para monitorar e controlar o robô virtual em tempo real.

1.3 Contribuições

Este trabalho visa deixar para os próximos estudantes os arquivos utilizados na sua realização, disponibilizando uma nova abordagem para o ensino de controle industrial e integração de sistemas. As principais contribuições podem ser citadas como:

Melhor compreensão dos conceitos de automação industrial, integração de sistemas e protocolos de comunicação, por meio da aplicação prática em um ambiente virtual.

Desenvolvimento de habilidades práticas em programação de controladores lógicos programáveis (CLPs), utilizando a norma IEC 61131-3, e na criação de interfaces de supervisão industrial.

Validação de estratégias de controle e supervisão em um ambiente virtual, permitindo a experimentação sem riscos de danos ao hardware real.

Proposta de uma metodologia de ensino que integra simulação robótica, controle industrial e supervisão, promovendo uma aprendizagem mais completa e contextualizada para os alunos de engenharia mecatrônica.

1.4 Organização do trabalho

No presente documento, a estruturação se deve a uma subdivisão em diferentes seções e subseções, cada uma com seu objetivo definido.

O capítulo 1 visa apresentar o problema da falta de recursos disponíveis para o estudo da automação industrial e, com isso, apresentar uma solução: a utilização de modelos digitais para substituir os custosos testes em equipamentos reais. São apresentados também os objetivos buscados, incluindo, mas não se limitando, às etapas necessárias para a conclusão básica do projeto, e por último a contribuição do trabalho, que disponibilizará os estudos realizados e os arquivos para que futuros estudantes possam usá-los como exemplo didático e de estudo.

No capítulo 2, são apresentados os principais fundamentos para a realização do projeto, sendo eles, os três softwares a serem utilizados: Webots, Codesys e InduSoft. Servindo de plataforma robótica, unidade de processamento industrial e interface de supervisão, respectivamente. Além disso, são apresentados os protocolos de comunicação utilizados para a integração entre os softwares, Modbus TCP e OPC UA, e a linguagem utilizada para a criação de um programa que realize a ponte de dados entre o Webots e o Codesys, Python.

No capítulo 3, tem-se o desenvolvimento do projeto, desde sua criação como um modelo na plataforma de simulação Webots, passando pela comunicação com o Codesys, até a integração final com o InduSoft, sendo citados os métodos utilizados para tais comunicações, e os problemas e soluções encontradas durante o avanço do projeto.

No capítulo 4, são apresentados os resultados obtidos durante o desenvolvimento do trabalho, incluindo a validação do modelo robótico, a análise da comunicação entre os sistemas e a avaliação da interface de supervisão.

No capítulo 5, são discutidos os resultados obtidos, comparando-os com os objetivos iniciais e destacando as contribuições do trabalho para o ensino de automação industrial.

2 DESENVOLVIMENTO

Tendo em vista a profundidade do assunto abordado e os conhecimentos utilizados para a realização do projeto, tem-se a necessidade de apresentar os principais pontos, conceitos e ferramentas utilizadas durante os estudos.

2.1 Simuladores

Tendo como base o objetivo principal de integrar diversos sistemas para a criação de uma plataforma de automação, faz-se necessário a utilização de simuladores para a criação de um ambiente virtual que permita a integração dos sistemas e a realização de testes sem a necessidade de um ambiente físico. Para isso, foram escolhidos três simuladores que comporão as diferentes partes e aspectos do trabalho.

2.1.1 Webots

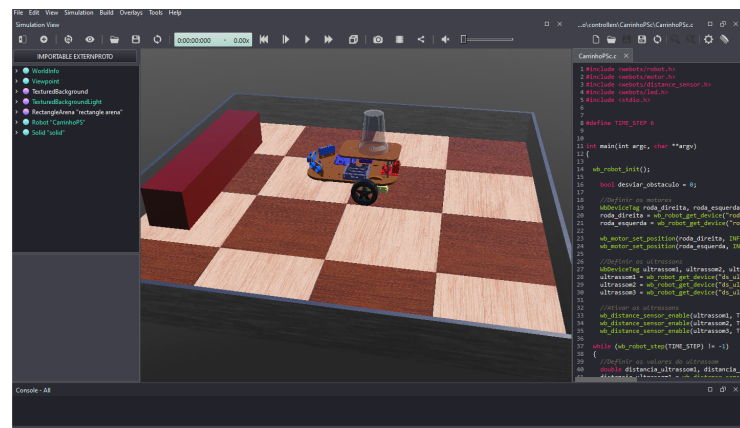


Figura 2.1: Logo do Webots e seu Espaço de Simulação.

Um simulador de robôs 3D que permite a criação de ambientes virtuais e a simulação de robôs em diferentes cenários. Com ele, é possível desde a montagem de diversos componentes no ambiente de simulação até a programação destes componentes para a realização de tarefas específicas. O simulador é capaz de simular diversos tipos de sensores e atuadores, permitindo a criação de componentes complexos e a realização de testes em diferentes condições.

Com suas diversas ferramentas e recursos, ele se torna um substituto do componente físico desejado, podendo ser utilizado para diversos tipos de testes controlados, para a obtenção de dados e informações relevantes. Segundo [Michel, 2004], a plataforma é segura e validada para ser utilizada em pesquisas e estudos de robótica, tendo seus cálculos e variáveis congruentes com testes realizados na realidade.

Para [Magrin et al., 2020], a utilização de um substituto digital para obter informações de dados físicos e dinâmicos de um sistema, não constitui perda ou diminuição da qualidade do estudo, sendo possível obter resultados satisfatórios e confiáveis para a realização de projetos e aplicações práticas. Apesar de o projeto digital não estar conectado a nenhuma unidade física para tal experimento, não sendo suficiente para se denominar *gêmeo-digital*, a modelagem e simulação da situação apresentada caracteriza a pesquisa como *Comissionamento virtual*, garantindo sua qualidade e confiabilidade.

A versão utilizada foi a *Webots R2025a* que, superiormente às versões anteriores, apresenta a utilização de um melhor formato de arquivo para seus componentes, o *.DAE (COLLADA)*. Este formato permite carregar geometrias complexas, materiais texturizados e dados de animação, com maior nível de compactação que os formatos de arquivos usados em versões anteriores e ultrapassadas *.WRL(VRML)*. Tal versão com os arquivos no devido formato gera um programa mais leve e com maior desempenho, tanto de velocidade quanto computacionalmente, permitindo que mesmo processadores com menor capacidade de processamento consigam executar simulações que não suportariam ou que teriam dificuldades com os formatos antigos.

2.1.2 Codesys

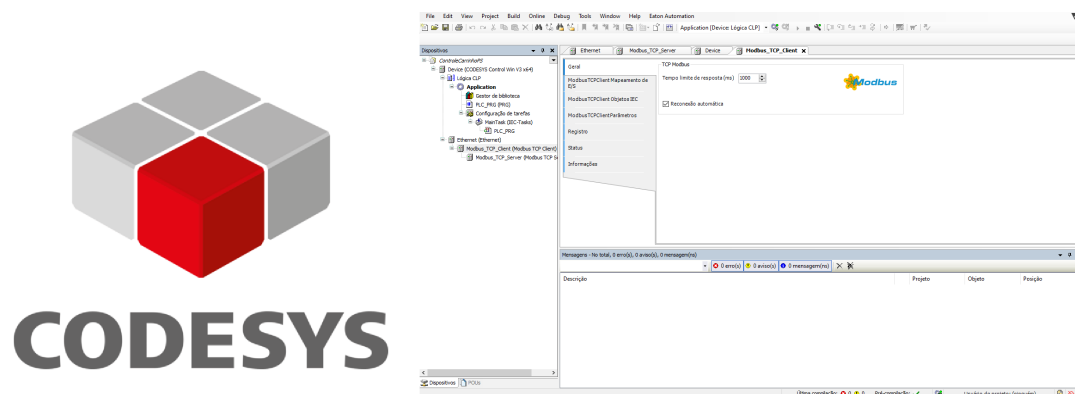


Figura 2.2: Logo do Codesys e seu Ambiente de Desenvolvimento.

Como uma plataforma de desenvolvimento de automação, é utilizado principalmente para a programação de diversos tipos de controladores lógicos programáveis (CLPs), sendo compatível com diversos fabricantes e modelos de CLPs. Ele também, oferece recursos de visualização de projetos em Interface Homem-Máquina (HMI), compatibilidade com redes de comunicação industrial e bibliotecas para outros tipos de aplicações, como controle de movimento, segurança e monitoramento de energia.

O Codesys como um SoftPLC, segue a norma internacional de padronização de linguagens de programação para CLPs, a *IEC 61131-3* [Lewis, 1998], que define

cinco linguagens de programação para CLPs, sendo elas: Ladder (LD), Texto Estruturado (ST), Diagrama de Blocos de Função (FBD), Lista de Instruções (IL) e Diagrama Sequencial de Função (SFC). Destas, apenas as linguagens *Ladder* e *Texto Estruturado* serão abordadas, por se tratarem das linguagens mais utilizadas e diretamente relacionadas com o conteúdo ministrado na instituição de ensino responsável pelo projeto.

Para o projeto, foi utilizada a versão *XSOFT-CODESYS V3.5.19 BF1*, uma versão já customizada pelo fabricante, vindo de fábrica com diversos recursos e bibliotecas já instaladas, como a biblioteca de comunicação falada na seção 2.2.1, Modbus TCP, e a biblioteca de comunicação falada na seção 2.2.2, OPC UA.

Para a comunicação entre o Codesys e o Webots, foi utilizado a linguagem Python, por ser simples, fácil de implementar, e por já possuir uma biblioteca que comporta a utilização da comunicação via Modbus.

2.1.3 InduSoft

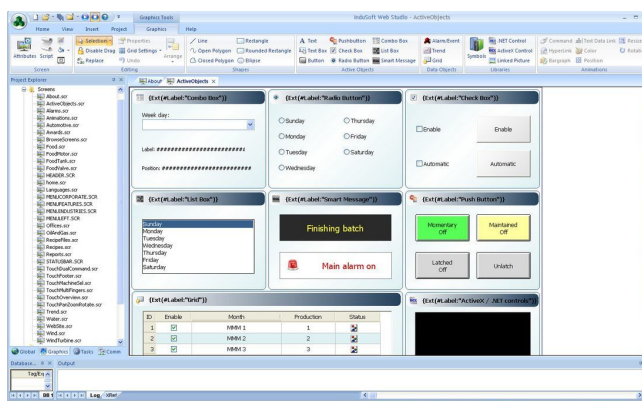


Figura 2.3: Logo do InduSoft e sua Área de Trabalho.

Um software de supervisão e controle, baseando-se em SCADA/HMI, permite a criação de interfaces gráficas e telas de operação para a visualização e controle de processos, sendo compatível com diversos protocolos de comunicação, como Modbus, OPC, BACnet, entre outros. Também coleta dados de CLPs, sensores e outros dispositivos, permitindo a análise e monitoramento de processos em tempo real. Além disso, possui recursos de alarmes, relatórios e gráficos, permitindo a criação de sistemas de supervisão e controle completos.

A marca foi adquirida pela Wonderware, que depois foi integrada ao grupo AVEVA, assim o sistema supervisório, agora foi renomeado de *AVEVA Edge*, mas para fins de estudo continuará sendo chamado pelo nome mais conhecido no meio industrial, InduSoft.

É baseado em softwares SCADA (Supervisory Control and Data Acquisition) e HMI (Human-Machine Interface), sendo assim, age como uma interface entre o operador e o processo, tanto no hardware de um componente físico, quanto no software de um componente virtual. Criando assim, uma ponte facilitadora entre o usuário e o sistema.

2.2 Comunicação

Para que os diversos sistemas venham a se comunicar, é necessária a utilização de protocolos de comunicação, conjuntos de regras e padrões que permitem a troca de informações entre diferentes dispositivos e sistemas. Dentre esses protocolos, foram escolhidos dois que se destacam em suas respectivas funções.

2.2.1 Modbus TCP

O Modbus é um protocolo que transporta dados entre dispositivos eletrônicos, via links seriais ou redes IP. Nele, um dispositivo central comumente chamado de 'Mestre/Cliente', faz solicitações, sendo elas perguntas ou ordens de envios de dados, e os dispositivos que recebem esses comandos, chamados de 'Escravo/Servidor', respondem aos requerimentos[WEG, 2024a]. Ele possui um protocolo simples e aberto, sendo livre de taxas e é compatível com uma enorme gama de dispositivos de diversos fabricantes, sendo popular na indústria.

Originalmente foi criado para ser transportado por cabos seriais, mas com o avanço das comunicações passou a ser adaptado para redes de internet industrial (Ethernet). Assim, as mensagens enviadas pelo protocolo passaram de usar um ID referente ao dispositivo a que se destina a mensagem, para serem enviadas dentro de pacotes de rede padrão da internet (pacote TCP/IP).

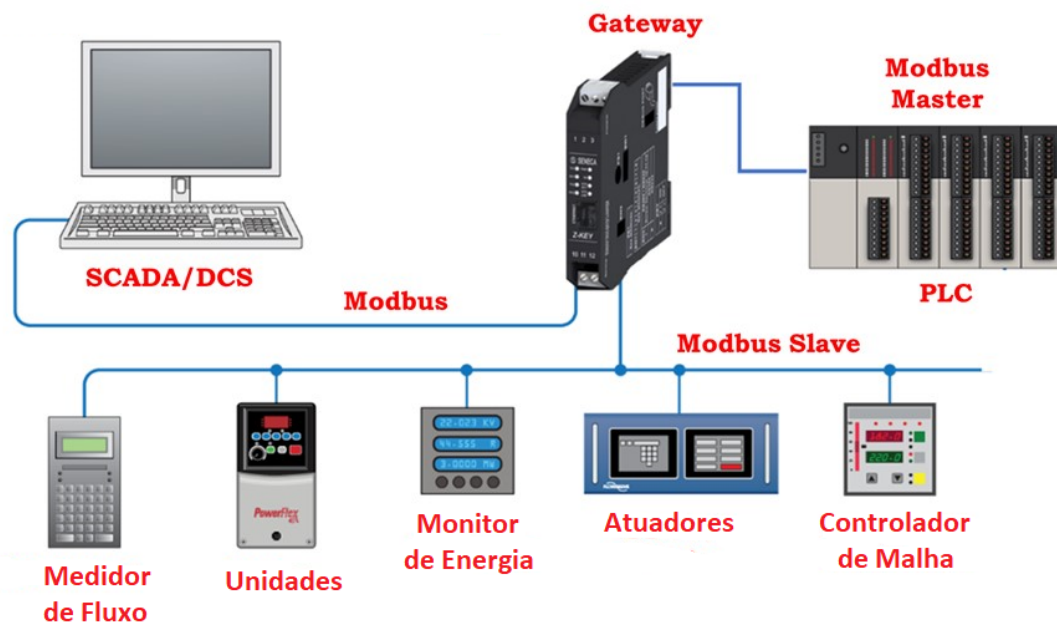


Figura 2.4: Protocolo de trabalho do Modbus.

Como visto na imagem 2.4, o Modbus funciona como a conexão entre o cliente e o servidor, ou entre o PLC 'Mestre' e os atuadores e sensores 'Servos', onde ele solicita informações, e os dispositivos na outra ponta respondem com os dados que obtiveram. Neste caso, o Medidor de Fluxo envia dados de vazão; a 'Unidades' (Inversor de Frequência) envia a condição do motor; o Monitor de energia envia dados de tensão, corrente e/ou consumo; o Atuador e Controlador de Malha enviam variáveis de leitura. Todos são conectados por um *Gateway* que traduz os comandos digitais do PLC para o formato serial utilizado pelos componentes, e depois o contrário, quando as informações estão sendo devolvidas ao PLC.

2.2.2 OPC UA

Anteriormente chamado *OPC*, foi criado para ser um idioma comum entre máquinas e permitir a comunicação entre dois dispositivos distintos e de marcas diferentes, assegurando que qualquer software se comunique com qualquer hardware. Ele funcionava como um tradutor intermediário que necessitava de um programa próprio para ser utilizado pela máquina, além de possuir dependência do Windows. Mas, com o avanço da indústria 4.0 e a necessidade de uma comunicação facilitada entre dispositivos, o protocolo, foi redesenhado em *OPC UA (Unified Architecture)*. Utilizando protocolos de rede modernos (TCP/IP, HTTP, entre outros), ele unificou todos os processos em um único ambiente, uma arquitetura unificada, além de não ser mais restrito pelo sistema operacional. Sendo assim, ele consegue enviar cadeias de dados complexas com diversas informações, não estando mais restrito a uma única informação por solicitação.

Ele funciona gerando um servidor (OPC Server)[WEG, 2024b] que se conecta a diversos dispositivos e disponibiliza os dados para os conectores (dispositivos conectados). Assim, softwares externos podem ler e escrever dados nos equipamentos sem a necessidade de conhecer todas as informações do hardware para o qual se está comunicando.

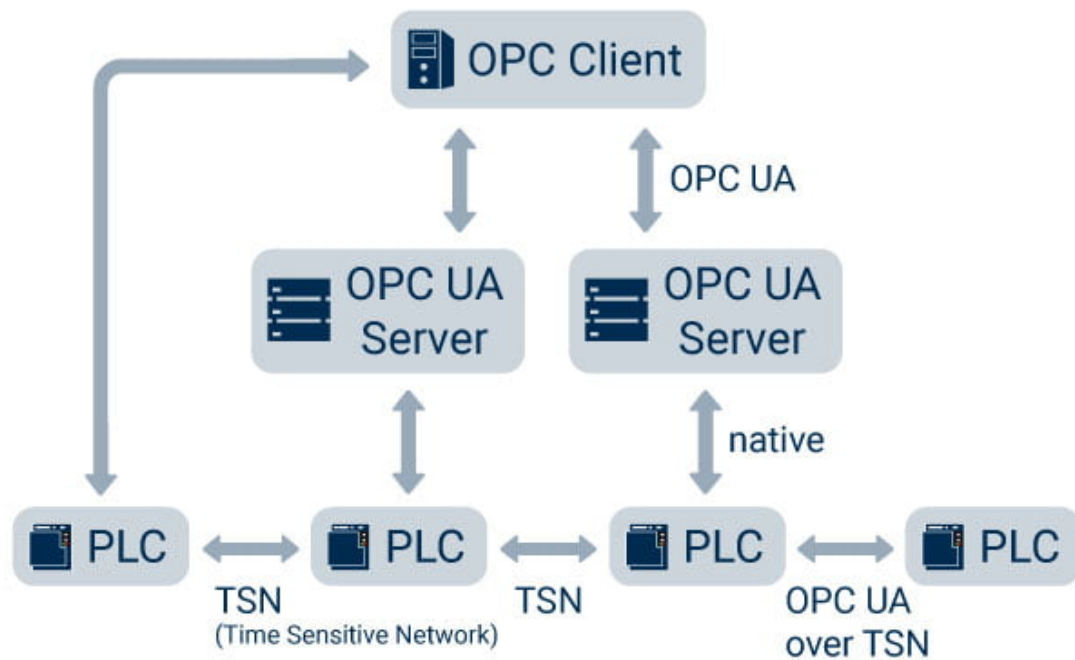


Figura 2.5: Aruquitetura do sistema OPC.

Na imagem 2.5, é visível o Cliente OPC representando um software de nível supervisor, como o InduSoft, ele se conecta padronizadamente com vários servidores usando o protocolo OPC UA, eles por sua vez, se conectam aos PLCs de base, traduzindo as informações geradas pelo dispositivo para um padrão universal que seja entendível pelo OPC Cliente.

2.3 Máquina

Com base em um projeto que os resultados específicos, ou a própria experiência de utilização dos simuladores, podem sofrer pequenas variações dependendo da máquina utilizada. É estabelecido as configurações da máquina do próprio autor do referido trabalho para comparação. Uma máquina de 2012 com uma única atualização de processador (A6-7400) e de armazenamento (466 GB HDD Toshiba):

- Placa mãe: A68HM-E33 (versão 9.0)
- Processador: AMD A10 PRO-7800B

- Placa de video: AMD Radeon R7 Graphics (integrada)
- Ram: 12,0 GB
- Armazenamento: 447 GB SSD Kingston

3 PROJETOS E DESENVOLVIMENTOS METODOLÓGICOS

Visando a integração dos três softwares para gerar um ambiente de automação industrial que possa fornecer aos alunos do CEFET-MG, principalmente para os que estão cursando automação industrial, um ambiente digital que possa ser modificado e usado como exemplo na sala de aula, foi primeiramente arquitetado e criado o modelo 3D do robô que servirá como teste e parâmetro para todas as etapas seguintes do projeto: o *Carrinho do Projeto Sementes 2020*.

3.1 Modelagem do Robô no Webots

Para a modelagem, foi utilizado o projeto anterior do carrinho já montado pelo autor do trabalho [Silva, 2020]. Consistindo em um modelo criado no software *SolidWorks* de 31 partes distintas. Cada peça possuía um número variável de outras partes modeladas na composição da mesma, além de haver elementos que foram utilizados mais de uma vez para a confecção do projeto, resultando em um modelo expandido com 304 *nós* distintos.

Os *Nós* são a representação de cada elemento modelado dentro do software Webots, eles são subdivididos em diversas categorias, mas as mais utilizadas para a montagem do carrinho se basearam nos Sólidos (Solid) e Transformados (Transform).

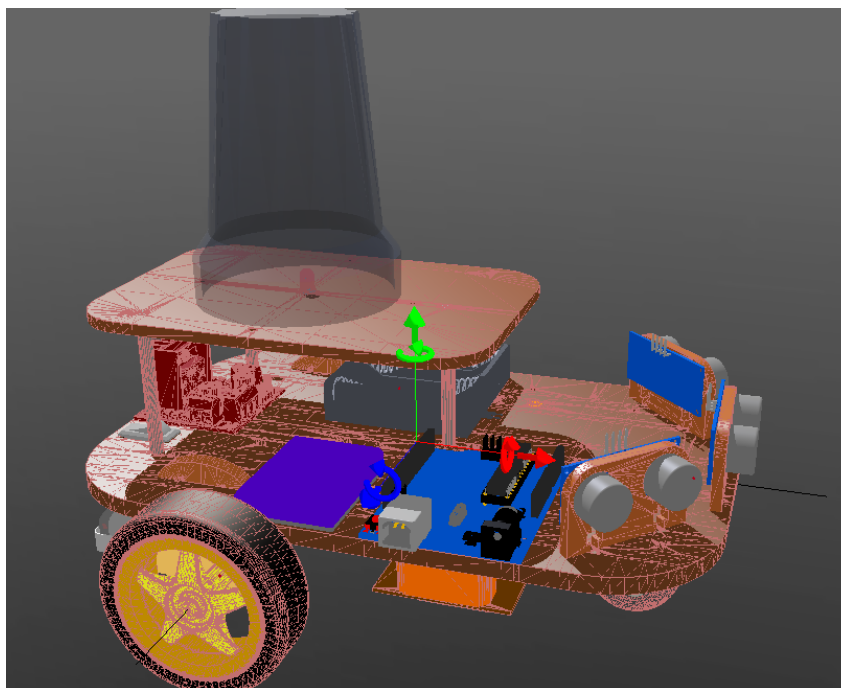


Figura 3.1: Modelagem do Carrinho Projeto Sementes 2020.

Com base na figura 3.1, observa-se o alto detalhamento nos seus mais diversos componentes, bem como a utilização dupla de algumas partes como: roda, suportes laterais, motores, ultrassons, entre outros. Com tal detalhamento, o programa muitas vezes precisava de uma máquina de maior potência ou de um sistema mais atualizado para realizar as simulações.

Tendo isso em vista, para a continuidade do projeto foi usada uma versão mais recente do software, como dito no capítulo 2.1.1, mudando da versão desatualizada R2020a, para a R2025a. Com essa nova versão, foi necessário atualizar o modelo criado, pois os arquivos anteriores com terminação em .WRL utilizados para a montagem não são mais aceitos pelo simulador.

Os arquivos antigos foram então convertidos em um novo formato, o COLLADA (.DAE), que além de ser mais rápido e compacto, permite que todo o programa rode com mais velocidade, e em máquinas de menor potência. Cada peça individual convertida foi então montada em um novo modelo, seguindo os passos já feitos para a modelagem de 2020.

Houve perdas de dados na conversão, resultando em modelos mais brutos e menos detalhados, o que só auxiliou o desempenho das simulações, pois cada peça agora possuía menos elementos em sua constituição.



Figura 3.2: Modelagem do Carrinho PFC 2026.

A nova reformulação da modelagem retirou elementos que não teriam importância para os testes com os outros simuladores, como os elementos de *Seguidor de Linha* e sensores complementares, além de reformular internamente os elementos

presentes em cada *nó*. Gerando, acima de tudo, um modelo mais leve e compacto, que possa ser usado em máquinas de pior desempenho.

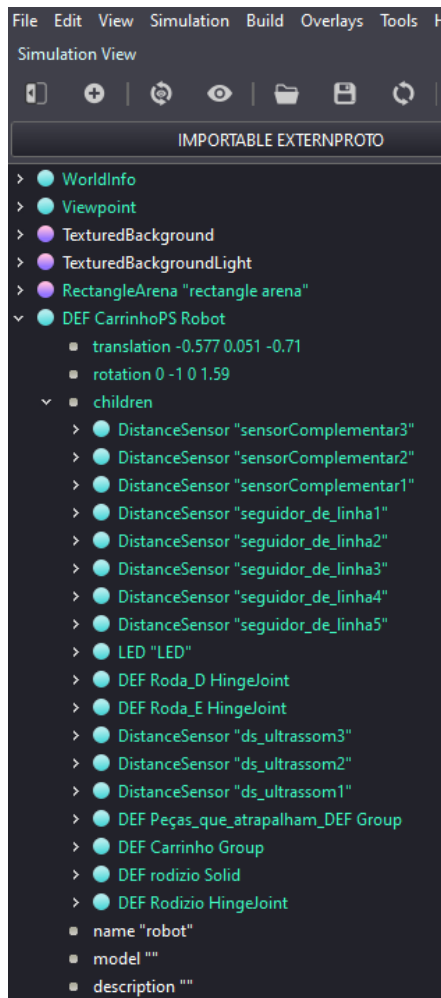


Figura 3.3: Árvore de Cenas 2020.

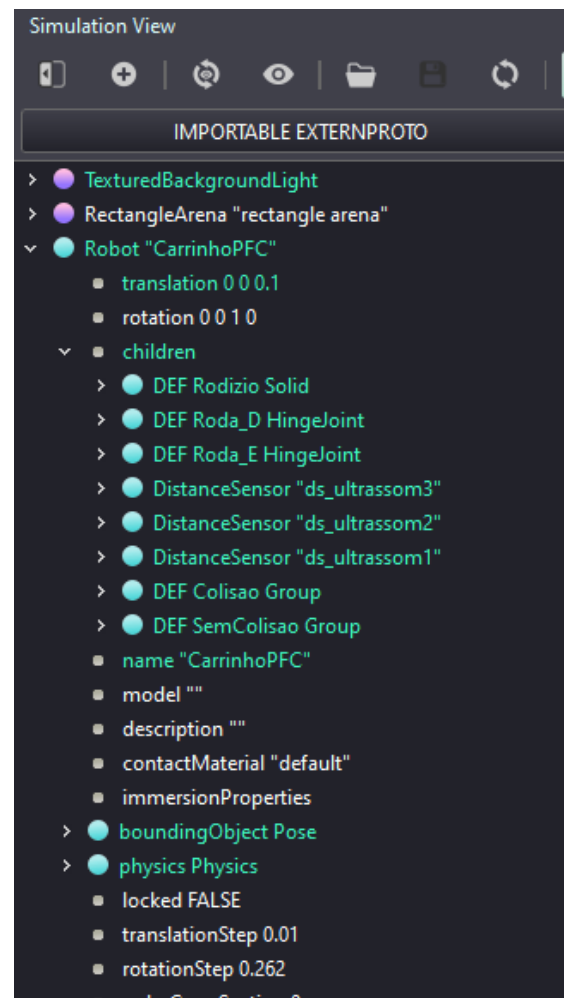


Figura 3.4: Árvore de Cenas 2026.

Para finalizar, um código em *C* e em *Python* foi gerado, o primeiro para referência, se baseando na primeira linguagem de programação que o modelo foi construído, e o segundo na linguagem que irá fazer a conexão com os demais softwares através da biblioteca Modbus.

Para referência, o código em anexo A foi o primeiro código gerado para testes de viabilidade.

3.2 Comunicação com Codesys

Após a finalização da montagem do robô de testes, é seguido para o software *CODESYS* para a criação de um ambiente que se comunique com o simulador robótico.

Assim, de acordo com [WEG, 2024a], foi implementada uma estrutura de Mestre/Servo entre os diferentes dispositivos. Com a criação de um dispositivo de *Ethernet* e subcriações dos elementos de cliente e servidor que representam a comunicação Modbus TCP/IP.

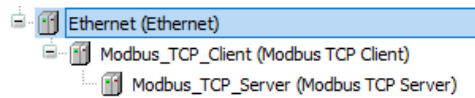


Figura 3.5: Configuração básica dos dispositivos Modbus.

Em seguida, é feita a configuração da conexão da rede, garantindo que os *IPs* do servidor Modbus e da rede de Ethernet sejam iguais. Este endereço IP e a porta atribuída a ele também serão utilizados para permitir a conexão com o código Python do simulador de robótica.

TCP Modbus	
Endereço IP do servidor	192 . 168 . 0 . 1
Tempo limite de resposta (ms)	1000
Porta	502

Figura 3.6: IP e Porta utilizados.

Para a utilização das variáveis que serão programadas posteriormente, tem-se a necessidade de criar um canal que as acomode através do dispositivo de servidor. No canal do servidor, foi configurado um *Write Multiple Registers (Código da Função 16)*, com um comprimento inicial de 2, buscando acomodar as variáveis básicas de aplicação dos motores do robô.

4 APLICAÇÕES E RESULTADOS

5 CONSIDERAÇÕES FINAIS

5.1 Conclusão

5.2 Sugestões de continuidade

REFERÊNCIAS BIBLIOGRÁFICAS

- Salvatore Cavaliere and Ferdinando Chiacchio. Analysis of opc ua performances. *Computer Standards & Interfaces*, 36(1):165–177, 2013. doi: 10.1016/j.csi.2013.06.004. URL <https://doi.org/10.1016/j.csi.2013.06.004>.
- Robert W. Lewis. *Programming Industrial Control Systems Using IEC 1131-3*, volume 50 of *Control Engineering Series*. Institution of Electrical Engineers (IEE), London, 1998. doi: 10.1049/PBCE050E. URL <https://doi.org/10.1049/PBCE050E>.
- Carlos E. Magrin, Lucas O. de Lima, André R. da Cruz, and Leandro B. de Almeida. Creating a digital twin as an open source learning tool for mobile robotics. *IFAC-PapersOnLine*, 53(2):17406–17411, 2020. ISSN 2405-8963. doi: <https://doi.org/10.1109/LARS/SBR/WRE54079.2021.9605457>. 21st IFAC World Congress.
- M. Marcos, E. Estévez, I. Sarachaga, and A. Burgos. Towards the conformance analysis of iec 61131-3 plc programming tools. *IFAC Proceedings Volumes*, 41(2):8446–8451, 2008. doi: 10.3182/20080706-5-KR-1001.01431. URL <https://doi.org/10.3182/20080706-5-KR-1001.01431>.
- Olivier Michel. Webots: Professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):39–42, 2004. doi: 10.5772/5618. URL <https://doi.org/10.5772/5618>.
- Cristiano Garcia Rosa, Joubert Vieira Santos, and Romeu Abrahão Pereira. Desenvolvimento de um sistema supervisor para planta didática de geração de potência. Technical report, Centro Brasileiro de Pesquisas Físicas (CBPF), Rio de Janeiro, 2019. URL https://www.academia.edu/74422433/Desenvolvimento_de_um_sistema_supervis%C3%B3rio_para_planta_did%C3%A1tica_de_gera%C3%A7%C3%A3o_de_pot%C3%Aancia.
- Christian Scheifele, Alexander Verl, and Oliver Riedel. Real-time co-simulation for the virtual commissioning of production systems. *Procedia CIRP*, 79:397–402, 2019. ISSN 2212-8271. doi: <https://doi.org/10.1016/j.procir.2019.02.104>.
- Getúlio A. Silva. Mini curso de modelagem Webots: Modelagem do robô da competição do Projeto Sementes. Vídeo no YouTube, 2020. URL <https://youtu.be/keCEhP2q56A?si=Q01s0vnIgecEsvwt>.
- WEG. *Application Note PLC500: Modbus TCP Client*. Jaraguá do Sul, 2024a. URL <https://static.weg.net/medias/downloadcenter/he8/h02/WEG-PLC500-Modbus-tcp-ApplicantionNote-10010166008-pt.pdf>. p. 12-18.

WEG. *Guia de Aplicação: OPC UA*. Jaraguá do Sul, 2024b.
URL [https://static.weg.net/medias/downloadcenter/h74/hd4/
WEG-PLC500-PLC410-OPC-UA-application-guides-10013132240-pt.pdf](https://static.weg.net/medias/downloadcenter/h74/hd4/WEG-PLC500-PLC410-OPC-UA-application-guides-10013132240-pt.pdf).

APÊNDICE A - TÍTULO DO APÊNDICE A

Primeira Seção

Subseção

Conteúdo...

ANEXO A - CODIGOS DO PROJETO

Codigo C - Modelagem Webots

```
1      #include <webots/robot.h>
2      #include <webots/motor.h>
3      #include <webots/distance_sensor.h>
4      #include <webots/led.h>
5      #include <stdio.h>
6
7
8      #define TIME_STEP 6
9
10
11     int main(int argc, char **argv)
12     {
13         wb_robot_init();
14
15         bool desviar_obstaculo = 0;
16
17         //Definir os motores
18         WbDeviceTag roda_direita, roda_esquerda;
19         roda_direita = wb_robot_get_device("roda_direita");
20         roda_esquerda = wb_robot_get_device("roda_esquerda");
21
22         wb_motor_set_position(roda_direita, INFINITY);
23         wb_motor_set_position(roda_esquerda, INFINITY);
24
25         //Definir os ultrassons
26         WbDeviceTag ultrassom1, ultrassom2, ultrassom3;
27         ultrassom1 = wb_robot_get_device("ds_ultrassom1");
28         ultrassom2 = wb_robot_get_device("ds_ultrassom2");
29         ultrassom3 = wb_robot_get_device("ds_ultrassom3");
30
31         //Ativar os ultrassons
32         wb_distance_sensor_enable(ultrassom1, TIME_STEP);
33         wb_distance_sensor_enable(ultrassom2, TIME_STEP);
34         wb_distance_sensor_enable(ultrassom3, TIME_STEP);
```

```

35
36 while (wb_robot_step(TIME_STEP) != -1)
37 {
38     //Definir os valores do ultrassom
39     double distancia_ultrassom1, distancia_ultrassom2, distancia_ultrassom3;
40     distancia_ultrassom1 = wb_distance_sensor_get_value(ultrassom1);
41     distancia_ultrassom2 = wb_distance_sensor_get_value(ultrassom2);
42     distancia_ultrassom3 = wb_distance_sensor_get_value(ultrassom3);
43
44
45     if(desviar_obstaculo == 0)
46     {
47         if(distancia_ultrassom1 < 700.0 || distancia_ultrassom2 < 700.0)
48         {
49             desviar_obstaculo = 1;
50         }
51         else if(distancia_ultrassom3 < 700.0)
52         {
53             desviar_obstaculo = 2;
54         }
55     }
56     else if (distancia_ultrassom1 > 900.0 && distancia_ultrassom2 > 900.0 && di
57     {
58         desviar_obstaculo = 0;
59     }
60
61     if (desviar_obstaculo == 1)
62     {
63         //Virar Direita
64         wb_motor_set_velocity(roda_direita, -1.5);
65         wb_motor_set_velocity(roda_esquerda, 3.0);
66
67     }
68     else if (desviar_obstaculo == 2)
69     {
70         //Virar Esquerda
71         wb_motor_set_velocity(roda_direita, 3.0);
72         wb_motor_set_velocity(roda_esquerda, -1.5);
73     }

```

```
74     else
75     {
76         //Ir para frente
77         wb_motor_set_velocity(roda_direita, 4.0);
78         wb_motor_set_velocity(roda_esquerda, 4.0);
79         printf("1E: %0.1f | 2C: %0.1f | 3D: %0.1f\n", distancia_ultrassom1, distanc
80     }
81
82 };
83
84
85 wb_robot_cleanup();
86
87 return 0;
88 }
```
